COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Oct 13, 2026	**DATE SUBMITTED:**	Oct 15, 2026
NAME:	Isaiah Benedict R. Garbosa Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 12 **Algorithmic Strategies**

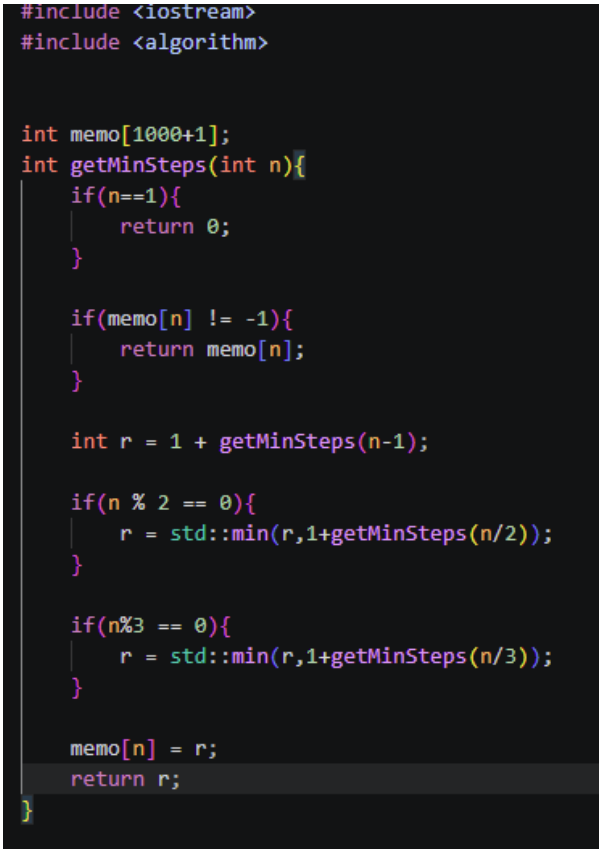
1. PROCEDURE OUTPUT

ILO A: Demonstrate an understanding of algorithmic strategies (Pre-Lab Review)

Strategy	Algorithm	Analysis
Recursion	**Tree traversals**	Recursion is when a function calls itself to solve a problem. Its idea is to break down the problem into smaller sub-problems until it is simple enough to solve directly. In our algorithms for the tree travels, it is done by using if statements. For example, our preOrder traversal prints a key first, then it checks if the first child is not a nullptr, calling the preOrder function, and similar for the sibling node, it checks if the sibling is not a nullptr, also calling the preOrder function.
Brute Force	**Linear search**	Brute force algorithm works by exhaustively searching through a problem and tests every single possibility until the condition is met or the search space is exhausted. Similarly, the linear search works by going through each element of an array until a key value is found or if it reaches the last element.
Backtracking	**Depth-First Search**	Backtracking refers to a strategy that builds up a solution incrementally and undoing or backtracking when it finds a path that does not have a possible solution. Similarly, Depth-First Search in graphs does the same. It starts at a chosen vertex then explores a connected path as deep as possible before backtracking once that path is done.
Greedy	**Selection sort**	A greedy algorithm works by building a solution piece by piece, taking optimal choices at each step and hoping to find a global optimum solution. Similarly, selection sort works almost the same. It makes locally optimal choices, to be exact, finding the current smallest value in the array. Once the smallest value is in place, move on to the next smallest value. It has the same principle as the greedy algorithm to never backtrack.

Divide-and-Conquer	**Merge sort**	Divide-and-Conquer is an algorithm where it breaks down a problem into smaller subproblems that are easier to manage and solve. Merge sort functions exactly like a Divide-and-Conquer algorithm since it splits an array first in the middle. Then, it further separates those subarrays. When subarrays are small enough, they are then sorted and arranged in place, resulting in a sorted array.

Table 12-1. Algorithmic Strategies and Examples

Screenshot	

```cpp
#include <iostream>
#include <algorithm>


int memo[1000+1];
int getMinSteps(int n){
    if(n==1){
        return 0;
    }

    if(memo[n] != -1){
        return memo[n];
    }

    int r = 1 + getMinSteps(n-1);

    if(n % 2 == 0){
        r = std::min(r,1+getMinSteps(n/2));
    }

    if(n%3 == 0){
        r = std::min(r,1+getMinSteps(n/3));
    }

    memo[n] = r;
    return r;
}
```

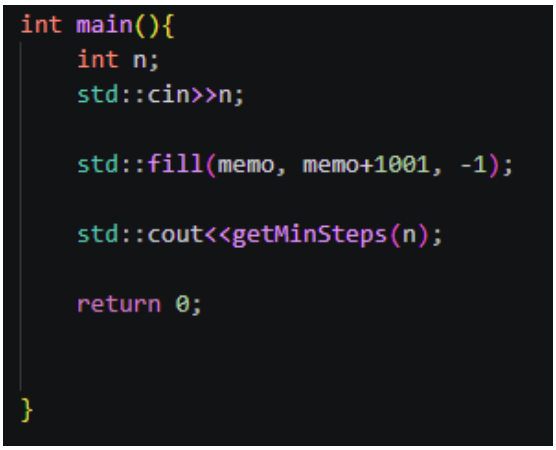
	
Analysis	The program finds the shortest way or the minimum number of steps to reduce a positive integer to 1. It does this by subtracting the positive integer by one, dividing by two, or dividing by 3 if possible. It uses recursion to backtrack and explore possible operations and min to choose the best possible option. Memoization allows the program to avoid previous calculations by storing previous results.

Table 12-2. Memoization Implementation

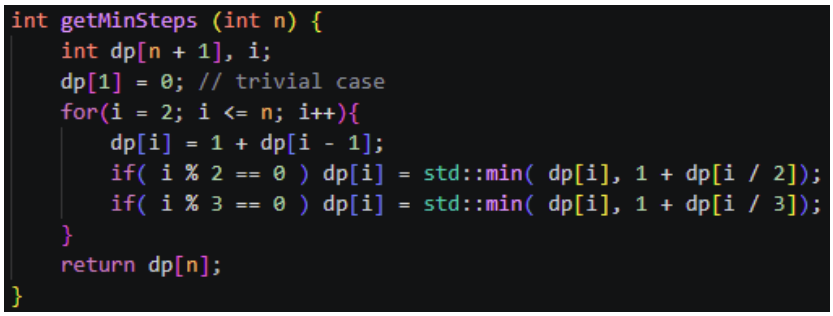
Screenshot	
Analysis	In Bottom-Up dynamic programming implementation, the program creates a tabulation where in steps or actions done are recorded and stored. This is used to review and reflect on decisions in order to solve the problem optimally. The difference when it comes to bottom-up is that, it builds from the bottom as the given name indicates. For this program, it starts with initializing an array that would be used in recording the steps it takes, and int i for the loop counter. The base case is dp[1] = 0 since 1 needs no steps to reach itself. Then the loop goes from 2 up to n, and for each i it first sets dp[i] to 1 + dp[i - 1], which is the subtract 1 option. After that, if i is divisible by 2, it compares the current value with 1 + dp[i / 2] and keeps the smaller one, and it does the same check for 3. Since every smaller number is already filled in by the time the loop gets to i, the program just looks those values up instead of recomputing them. In the end, dp[n] holds the minimum number of steps for n, so that's what gets returned. This runs in O(n) time because it's one loop with constant work each time, and it uses O(n) space for the array.

Table 12-3. Bottom-Up Dynamic Programming Implementation

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Create C++ code to implement various algorithmic strategies & ILO C: Create C++ code to solve problems using learned algorithmic strategies

Problem	Count the number of paths in a matrix with a given cost to reach the destination cell

Given an M × N integer matrix where each cell has a non-negative cost associated with it, count the number of paths to reach the last cell (M-1, N-1) of the matrix from its first cell (0, 0) such that the path has a given cost. We can only move one unit right or one unit down from any cell, i.e., from cell (i, j), we can move to (i, j+1) or (i+1, j).

Pseudocode

```
                    countPaths Function

FUNCTION countPaths(matrix, row, col, cost):
      IF row or col is outside the matrix:
            RETURN 0

      cost = cost - matrix[row][col]

      IF cost < 0:
            RETURN 0

      IF row and col are the last cell:
            IF cost == 0:
                  RETURN 1
            ELSE:
                  RETURN 0

      SET rightPaths = countPath(matrix, row, col + 1,
      cost)
      SET downPaths = countPaths(matrix, row + 1, col,
      cost)

      RETURN rightPaths + downPaths

END FUNCTION


                              MAIN
START
      SET matrix and target cost
      SET result = CALL countPaths function

      PRINT "Target cost: ", targetCost
      PRINT "Number of paths: ", result
END
```

Solving Problem by Hand

Using the given matrix:

4	7	1	6
6	7	3	9
3	8	1	2
7	1	7	3

If we look for paths that will result in a cost of 25, it will not be possible. The given example results on in

4 + 7 + 1 + 3 + 1 + 2 = 18

There aren't any other paths that will equal a cost of 25 if we only can move one to the right or one to the left. However, if we change the cost to 26, we can have some available paths.

FIRST PATH: 4 + 6 + 7+ 3 + 1 + 2 + 3 = 26
SECOND PATH: 4 + 7 + 1 + 3 + 1 + 7 + 3 = 26

Thus, there are 2 possible paths for 26 and 0 for 25.

Code

```
#include <iostream>

const int n = 4;

int countPaths(int matrix[n][n], int row, int col, int cost){
    if(row >= n || col >= n){
        return 0;
    }

    cost = cost - matrix[row][col];

    if (cost < 0){
        return 0;
    }

    if (row == n-1 && col == n-1){
        if(cost == 0){
            return 1;
        }
        return 0;
    }

    return countPaths(matrix, row, col + 1, cost) +
    countPaths(matrix, row + 1, col, cost);
}
```

```
int main() {
    int matrix[n][n] = {
        {4,7,1,6},
        {6,7,3,9},
        {3,8,1,2},
        {7,1,7,3}
    };

    int cost;

    std::cout<<"Input target cost: ";
    std::cin>>cost;


    std::cout<<"Target cost: "<<cost<<std::endl;
    std::cout<<"Number of paths: "<<countPaths(matrix, 0, 0, cost)<<std::endl;
}
```

Analysis

The function countPaths takes the parameters of a 2D array with their global size n set to 4, and variables row, col, and cost with an integer data type. The first if condition is to catch when row and col are out of bounds or their positions do not exist in the matrix. If they satisfy the condition of the if statement, then the function returns 0. The next line of code is to calculate the amount left after subtracting the cost it takes to occupy the position in the matrix that the function is currently on. Then the next if condition is to compare whether the cost has reached the negatives where it is smaller than 0, this is because the given matrix has non-negative cost associated with each cell, so the cost will not go back up. If the cost is less than 0, then the path ends without reaching the end, and the function returns 0. Then the last if condition will evaluate if the function has reached the end of the matrix. It checks if the row and col equal to n-1, which refers to the bottom right of the matrix. Within that condition exists another if statement, designed to check if the cost has reached 0, thus returning 1. Then, if the cost is not equal to 0, the function returns 0. Afterwards, the last lines of code within the function refers to the movement in which the function can take, moving only 1 unit right or down. This will call recursion on itself as it moves through the matrix, counting every path that works. This does not stop at the first match as it adds up the results from the right and down moves.

Demonstration

```
Input target cost: 25
Target cost: 25
Number of paths: 0
```

```
Input target cost: 26
Target cost: 26
Number of paths: 2
```

Conclusion - Garbosa
In this activity, I was able to learn about different algorithmic strategies and how they work. I learned about recursion, where the function calls itself to solve a smaller part of a problem, brute force, where it checks every possible option until it find the correct answer, backtracking, which explores a path and then backtracks if it doesn't find a valid solution, greedy algorithm, which chooses the best available option at each step without backtracking, and divide-and-conquer which divides a large problem into smaller subproblems, solves them separately, and then combines their results to solve the problem. I also was able to apply most of these strategies like applying recursion in the supplementary activity to count the number of possible paths in a matrix with a given cost. Furthermore, I learned how memoization and bottom-up dynamic programming solves problems by storing previous results and avoiding repeated calculations. Overall, I believe that the activity was able to help me understand how different algorithmic strategies can be used to solve problems. I believe that I was able to learn and apply these concepts but I am still having a hard time to process or master some of the concepts like the greedy algorithm and brute force algorithm. I will try to practice and learn to apply these strategies in a more realistic situation.

Conclusion - Vizcayno
The activity discussed algorithmic strategies like recursion, brute force, backtracking, greedy, divide & conquer, and dynamic programming. All of which are approaches to solving problems. Some of these algorithmic strategies are already in many familiar forms such as tree traversals, linear and binary search, and sorting algorithms. It goes to show how useful these strategies are in solving various problems. In the supplementary, we get to experience how dynamic programming works with the given problem. It proposed a problem where you must create a program that counts the number paths available with a given cost using the given matrix. To solve the given problem, we used plain recursion that brute forces the possible answers. While this may be less efficient, it gets the job done. Although if we had enough time, we could have made something more efficient that utilizes dynamic programming. As for me, I think that I still need more time to process how dynamic programming works as it's a concept I still struggle to understand. I also need to manage my time in a way I could contribute more to the activities.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used for correcting grammar in analysis and conclusions.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.